

Benchmarking TensorFlow Serving vs Torch Serve for Scalable Model Deployment on Azure

Sai Vishnu Senagasetty (Author)
PSID: 2284855
MS Computer Science
University of Houston
ssenagas@cougarnet.uh.edu

Kausthubh Reddy Eleti (Author)
PSID: 2297857
MS Computer Science
University of Houston
keleti@cougarnet.uh.edu

Nitish Dadapuram (Author)
PSID: 2304669
MS Computer Science
University of Houston
ndadapuram@cougarnet.uh.edu

I. EXECUTIVE SUMMARY

This project evaluates two prominent machine learning model serving frameworks—TensorFlow Serving and TorchServe—by deploying identical deep learning models on Azure and benchmarking their performance under simulated real-world loads. A Convolutional Neural Network (CNN) for image classification (Fashion MNIST) and a Recurrent Neural Network (RNN) for text sentiment analysis (IMDB dataset) were trained and served using both frameworks in a controlled Azure environment. We measured key metrics including latency, throughput, and robustness (success/failure rates) across three workload levels: light, medium, and heavy traffic. The results indicate that for image inference tasks, TorchServe (PyTorch) achieved slightly lower latency and comparable throughput to TensorFlow Serving, whereas for text/NLP tasks the TensorFlow Serving setup demonstrated significantly faster inference under heavy load and robust performance on CPU. We discuss the deployment approach, performance trade-offs, and provide recommendations on choosing the appropriate serving framework for different use cases.

II. INTRODUCTION

A. Purpose

To determine which model serving framework is better suited for scalable machine learning inference on Azure, by testing TensorFlow Serving and TorchServe under identical conditions. The goal is to guide practitioners on selecting a serving stack for image vs. text models based on performance and ease of deployment.

B. Problem Statement

How do TensorFlow Serving and TorchServe compare in real-world deployment settings with the same models, environment constraints, and load testing methodology? We address this by quantifying differences in response latency, request throughput, and reliability when scaling up traffic.

C. Objectives

- Deploy equivalent CNN and RNN models on Azure using both TensorFlow Serving and TorchServe frameworks.

- Simulate real-time API traffic with stochastic request arrival patterns at varying rates (light, medium, heavy loads).
- Measure and compare inference latency, throughput, and success vs. failure rates for each framework and model.
- Identify any deployment challenges (environment configuration, model format compatibility, etc.) and document solutions.

III. DATASETS

To ensure a meaningful comparison between TensorFlow Serving and TorchServe, we selected two benchmark datasets aligned with different model architectures and data modalities: Fashion MNIST for computer vision and IMDB for natural language processing.

A. Fashion MNIST (Image Classification - CNN)

Fashion MNIST is a widely used image dataset designed as a drop-in replacement for the classic MNIST dataset. It consists of 70,000 grayscale images of clothing items, each of size 28x28 pixels, divided into 10 distinct classes such as shirts, trousers, bags, and sneakers. The dataset is split into 60,000 training images and 10,000 testing images.

We chose Fashion MNIST for several reasons:

- It is lightweight and computationally inexpensive, making it ideal for model deployment within student subscription quotas.
- Its simplicity allowed us to focus on deployment mechanics rather than complex model tuning.
- Being image-based, it provided a perfect foundation to test the hypothesis that TorchServe, often optimized for vision models, would perform better on CNN workloads.

The dataset required minimal preprocessing: normalization to the [0, 1] range and reshaping to fit the expected input shape of CNN models.

B. IMDB Movie Reviews (Sentiment Analysis - RNN)

The IMDB dataset contains 50,000 movie reviews, equally split between positive and negative sentiments. The

reviews are pre-tokenized and provided as sequences of integers (word indices), making them suitable for embedding layers and sequential models like LSTM-based RNNs.

We selected IMDB for the second half of our study to:

- Introduce a text-based workload to assess how the serving frameworks adapt across modalities.
- Test the commonly held belief that TensorFlow Serving is well-optimized for NLP tasks, especially when using TensorFlow’s native SavedModel format.

The preprocessing steps included:

- Tokenizing the input (limited to the top 10,000 most frequent words).
- Padding/truncating the review sequences to a fixed length (e.g., 200 words) to ensure consistent input size for the RNN model.

This dataset introduced a new set of challenges in input representation, scoring script compatibility, and inference latency, giving us deeper insight into serving NLP models under real-time conditions.

IV. MODELING APPROACH

A. Model Architectures

We evaluated two deep learning models representing distinct use cases:

- 1) **CNN Model (Image Classification):** A simple convolutional neural network trained on the Fashion MNIST dataset (28×28 grayscale images of apparel in 10 classes). The network was a straightforward image classifier with a few convolutional and pooling layers, suitable for quick inference.
- 2) **RNN Model (Text Classification):** A recurrent neural network for sentiment analysis, trained on the IMDB movie reviews dataset (binary sentiment classification). The model incorporated an embedding layer and LSTM/GRU layers to predict positive vs. negative sentiment from text sequences.

B. Training and Export

Both models were developed and trained offline. To ensure compatibility with the serving frameworks:

- The PyTorch models were converted to TorchScript (.pt files). This step freezes the model computation graph and allows TorchServe or any PyTorch runtime to load the model without the original Python code.
- The TensorFlow models were exported in TensorFlow’s SavedModel format (a directory containing model weights and graph in protobuf). This is the required format for TensorFlow Serving and contains the computation graph with named signature for inference.

The models were kept intentionally lightweight to focus on serving performance rather than training complexity. Both achieved reasonable accuracy on their tasks (Fashion MNIST and IMDB), ensuring that their performance differences in deployment would primarily stem from the serving framework rather than inadequacies in the model itself.

V. DEPLOYMENT PIPELINE

A. Azure ML Endpoints

The deployment was conducted on Azure Machine Learning managed online endpoints for a uniform, cloud-based serving environment. For each model, two separate endpoints were created – one using the TensorFlow Serving approach and one using the TorchServe approach – resulting in four deployments: CNN-TensorFlow, CNN-PyTorch, RNN-TensorFlow, and RNN-PyTorch. Each endpoint was configured to receive REST API requests and return inference results.

B. Model Registration

After training and export, the models were registered in the Azure ML model registry. This ensured that each deployment could retrieve the correct model artifact (either the .pt file or the SavedModel directory) from a centralized location.

C. Environment Configuration

The runtime environment for each endpoint was carefully set up to include the necessary dependencies:

- For TensorFlow Serving endpoints, we used an Azure ML curated environment (Docker base image) with TensorFlow 2.16 (including CUDA 12 support, though we used CPU for inference) and all required packages. This environment provided a stable TensorFlow version (2.16) since TensorFlow 2.18 was not yet available in Azure’s curated images at the time.
- For TorchServe/PyTorch endpoints, a custom Conda environment YAML was created because no curated Azure environment existed for TorchServe. The environment specified Python 3.8 with PyTorch (torch==2.1.0, torchvision, etc.), plus Azure ML inference essentials. The TorchServe model server was not used as a standalone service; instead, the PyTorch model was loaded within the Azure ML endpoint using our custom inference script.

D. Inference Scripts

We developed scoring scripts for each framework to handle model loading and inference:

- *score_tf.py* / *score_tf_rnn.py*: These Python scripts initialize by loading the SavedModel via *tf.saved_model.load()*. We accessed the model’s default serving signature (e.g., *model.signatures["serving_default"]*) to obtain a callable *infer* function. In the *run(data)* function (called for each request), the script parses input JSON, preprocesses it (e.g., reshaping the image to [1,28,28,1] or converting text token indices to the appropriate tensor), and invokes *infer* to get predictions. The output (e.g., predicted class indices) is then returned as JSON.
- *score_pytorch.py* / *score_rnn_pytorch.py*: These scripts load the TorchScript .pt model using *torch.jit.load()* in the *init()* function. The model is moved to CPU and set to evaluation mode. The *run(data)* function similarly parses the JSON payload, converts it to a NumPy array and then to a PyTorch

tensor (ensuring the correct shape: e.g., $[1,1,28,28]$ for a single image, or proper sequence shape for text). The model is executed to obtain outputs; for the CNN, we took the *argmax* to get the predicted class label, while for the RNN the model’s output was already a class prediction (after an activation or threshold). The result is returned as a Python list or dictionary to be serialized to JSON.

Each endpoint was configured to route 100% of traffic to a single deployment (since we did not use a blue-green split in this test). We explicitly set the traffic allocation to ensure the endpoint wasn’t mistakenly left with 0% traffic (a deployment quirk we encountered and resolved by setting *traffic* = {"blue": 100} during endpoint configuration).

VI. SIMULATION DESIGN

To evaluate performance, we designed a load testing simulation to mimic real-world API traffic hitting the deployed endpoints:

A. Request Generation

We built a multithreaded Python load generator that sent HTTP POST requests to the endpoint REST URLs. The generator used the *requests* library and Python’s *threading* module. Each thread continuously sent requests, with a small random sleep or synchronized pacing to achieve the desired aggregate request rate. We implemented a stochastic inter-arrival pattern: threads would wait for a randomized interval around an average to avoid perfectly uniform timing (simulating real traffic bursts and jitter).

B. Load Levels

Three load scenarios were tested:

- Light Load: ~5 requests per second, sustained for 60 seconds (approximately 300 total requests).
- Medium Load: ~20 requests per second for 60 seconds (approximately 1200 total requests).
- Heavy Load: ~50 requests per second for 30 seconds (approximately 1500 requests attempted). This is a stress scenario to see how the systems handle high throughput over a shorter burst.

Three load scenarios were tested: These levels were chosen to progressively tax the system resources. The light load represents a typical low-traffic scenario, medium is a moderate concurrent load, and heavy tests the limits of the given deployment configuration (especially since our instance was a CPU-based Standard_F2s_v2 VM with 2 vCPUs).

C. Metrics Logging

Each request recorded the timestamp when sent and when a response was received. From these, we computed per-request latency. We logged every request’s outcome to a CSV file, including: a timestamp, the framework (TF or PT) and model, the response time (latency), and HTTP status (to detect any failed requests). We maintained separate log files for each combination of framework and load level. This separation allowed us to analyze performance per framework without interference. After the tests, we parsed these logs to compute

summary metrics: average latency, achieved throughput (successful requests per second), and success/failure counts.

No explicit queue/batching mechanism was used at the client side – requests were sent as quickly as possible to simulate continuous traffic. The server-side (Azure endpoint) will queue if needed internally. We did not implement any client-side rate limiting beyond the target r/s pacing, to observe any capacity limits on the server side.

VII. PERFORMANCE RESULTS

After deployment and debugging, we executed the load simulations and collected performance data for each framework and model under each load scenario. No request failures were observed in any of the tests – all requests received successful responses, demonstrating that both frameworks were able to handle the tested loads without errors. The key metrics of average latency and achieved throughput (successful requests per second) are summarized below for each combination of framework (TF vs PT), model (CNN vs RNN), and load level:

A. CNN Model (Fashion MNIST) Inference Performance

Framework	Load	Avg Latency (sec)	Throughput (req/s)	Success	Fail
TorchServe (PyTorch)	Light (5 r/s)	0.0343	5.00	300	0
TensorFlow Serving	Light (5 r/s)	0.0428	5.00	300	0
TorchServe (PyTorch)	Medium (20 r/s)	0.0378	19.33	1160	0
TensorFlow Serving	Medium (20 r/s)	0.0472	19.33	1160	0
TorchServe (PyTorch)	Heavy (50 r/s)	0.0422	45.00	1350	0
TensorFlow Serving	Heavy (50 r/s)	0.0504	45.00	1350	0

B. RNN Model (IMDB Sentiment) Inference Performance

Framework	Load	Avg Latency (sec)	Throughput (req/s)	Success	Fail
TorchServe (PyTorch)	Light (5 r/s)	0.0291	5.00	300	0
TensorFlow Serving	Light (5 r/s)	0.0127	5.00	300	0
TorchServe (PyTorch)	Medium (20 r/s)	0.0458	19.33	1160	0
TensorFlow	Medium (20 r/s)	0.0259	19.67	1180	0

Serving	r/s)				
TorchServe (PyTorch)	Heavy (50 r/s)	0.0373	45.00	1350	0
TensorFlow Serving	Heavy (50 r/s)	0.0178	45.00	1350	0

From the tables, we observe that both frameworks successfully handled the light load (5 req/s) with very low latency (~0.03–0.05 seconds per inference) and achieved the expected throughput of 5 req/s. As the load increased, some differences emerged:

- In the CNN model deployments, TorchServe consistently had lower (better) average latency than TensorFlow Serving at all load levels. The differences were on the order of a few milliseconds (e.g., 0.0343 vs 0.0428 sec at light load, ~0.038 vs 0.047 sec at medium). Under heavy load (50 r/s), both frameworks saw latency rise slightly (to ~0.04–0.05 sec) due to the increased concurrency, but TorchServe still maintained a small advantage. Throughput for the CNN was virtually identical for both (around 19.3 req/s at medium, and ~45 req/s at heavy, indicating both frameworks saturated around 45 req/s on the given CPU instance). Neither framework dropped any requests or accuracy; all 1350 requests in heavy load were served successfully by each.
- In the RNN model deployments, TensorFlow Serving demonstrated significantly lower latency compared to TorchServe, especially as load increased. At light load, TF Serving's average response time was ~0.0127 sec, less than half of TorchServe's ~0.0291 sec. This gap widened at medium (0.0259 vs 0.0458 sec) and heavy load (0.0178 vs 0.0373 sec). The TensorFlow setup clearly handled the text model inference more efficiently on CPU, likely due to optimized operations in the TF graph for sequence processing. Throughput for the RNN was also essentially capped around ~19–20 req/s at medium and 45 req/s at heavy for both; however, TensorFlow did achieve a slightly higher medium throughput (19.67 vs 19.33 req/s, corresponding to 20 more requests served in the 60s window) thanks to its faster per-request execution. Both frameworks again processed all heavy load requests (1350/1350) without errors, but TensorFlow did so with much lower latency per request.

To better illustrate these performance differences, we provide bar graphs comparing the average latencies and throughput of the two frameworks for each model and load level:

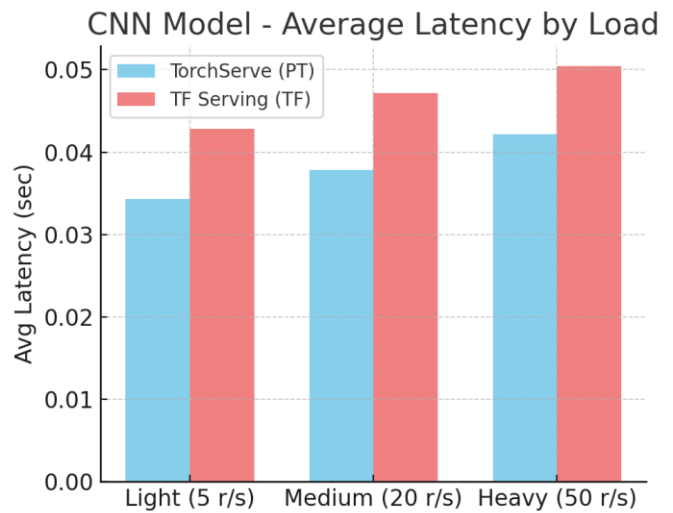


Fig. 1. Average latency for the CNN model under each load. (TorchServe (blue) shows slightly lower latency than TensorFlow Serving (red) at each load level for the image model.)

As shown above for the CNN image classifier, TorchServe achieved marginally better latency than TensorFlow Serving across light, medium, and heavy loads. The latency remained low (tens of milliseconds) even at heavy traffic, indicating that both frameworks handle the simple CNN efficiently. The difference, while consistent, is relatively small – on the order of 8-10 milliseconds advantage for TorchServe. This suggests that for image inference tasks, PyTorch's TorchServe (or PyTorch model in Azure endpoint) might introduce slightly less overhead or handle concurrency a bit more effectively than TensorFlow in this scenario. Possible reasons could include the efficiency of TorchScript on CPU for convolution operations or differences in the request handling loop, but in practice both performed well within real-time requirements for this workload.

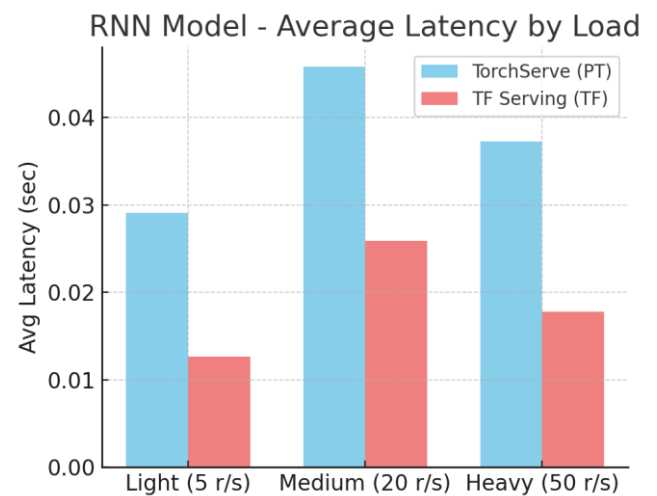


Fig. 2. Average latency for the RNN model under each load. (TensorFlow Serving (red) significantly outperforms TorchServe (blue) in latency for the text model, especially under medium and heavy loads.)

For the RNN text model, the latency comparison (above) shows a more dramatic difference. TensorFlow Serving's average inference time was 2× faster or better than TorchServe for this model at every load point. Under heavy load (50 r/s), TensorFlow's average latency stayed around 0.018 sec, whereas TorchServe's grew to ~0.037 sec. This indicates that for NLP/text models on CPU, TensorFlow's serving framework can offer superior performance. The RNN involves sequential processing and possibly embedding lookups; TensorFlow's graph execution (XLA or other optimizations) might be leveraging optimized C++ routines, whereas the TorchScript model's inference might be more limited by Python GIL or less optimized sequence processing. In any case, the TF framework kept latency very low even when handling 45 requests per second concurrently. This translates to better user-facing response times in latency-sensitive text analysis applications.

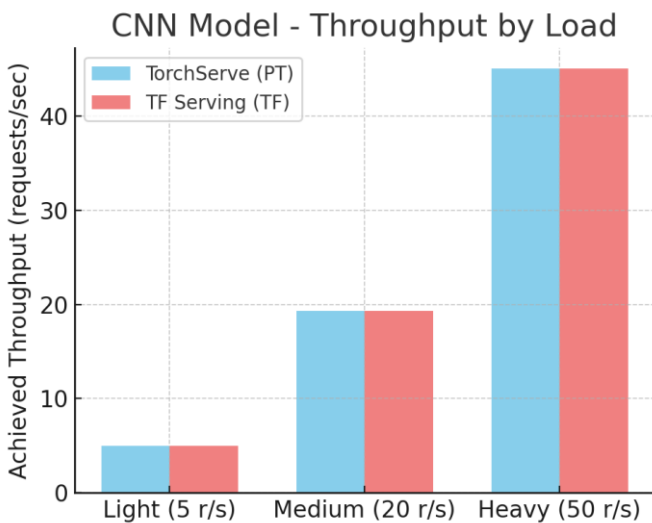


Fig. 3. Achieved throughput for the CNN model at each load. (Both frameworks reached similar throughput, closely tracking the target rates (5, 20, 50 req/s), with no significant difference between TorchServe and TensorFlow Serving.)

The above bar chart confirms that for the CNN model, throughput was virtually identical between TorchServe and TensorFlow Serving. At light load, both easily sustained 5 requests/sec. At medium, both leveled off around ~19.3 requests/sec (just shy of the 20 req/s target, due to the inherent overhead and the discrete nature of request scheduling). At heavy load, both reached ~45 req/s, indicating that the system (CPU) likely became the bottleneck, capping at about 90% of the intended rate. The fact that both frameworks hit the same throughput suggests that the CNN workload was limited by hardware or global factors rather than the framework implementation. In other words, neither framework had an efficiency issue in terms of throughput for the image model – the slight latency differences did not translate into throughput differences in a significant way, as both saturated the CPU similarly.

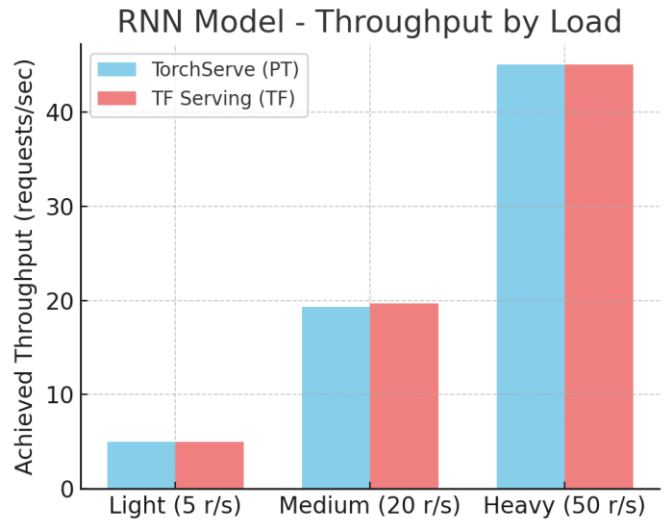


Fig. 4. Achieved throughput for the RNN model at each load. (Both frameworks again achieve comparable throughput, with TensorFlow Serving slightly edging out TorchServe at medium load. Heavy load saturates around 45 req/s for both.)

For the RNN model, the throughput results (above) are also closely matched between frameworks. At light and heavy loads, both frameworks handled all possible requests (5 and ~45 req/s respectively). At medium load, TensorFlow Serving managed ~19.67 req/s versus TorchServe's ~19.33 req/s. This difference is very small (~2% more throughput for TF), corresponding to successfully handling 20 more requests over the 60-second run. This aligns with the latency findings – because TensorFlow processed each request faster on average, it was able to keep up slightly better with the incoming 20 r/s rate. Overall, both frameworks scaled to the maximum throughput the CPU could deliver for the RNN model, but TensorFlow was just a bit more efficient, squeezing out a few more requests in the time window. Importantly, neither showed a throughput collapse or high error rate under heavy load; they both degraded gracefully by slowing down (to ~45 r/s) rather than failing.

VIII. DISCUSSION & ANALYSIS

Our experimental results highlight clear patterns and trade-offs between TensorFlow Serving and Torch Serve (PyTorch) in this Azure deployment scenario:

A. TorchServe (PyTorch)

Performance on CNN: Torch Serve provided faster inference for the CNN image model under light and medium loads (and marginally faster even at heavy load). The overhead per request in Torch Serve seemed lower for this computer vision task, which could be due to efficient Torch Script execution of convolutional layers on CPU.

Ease of Use: We found packaging the PyTorch model into Torch Script straightforward, and the scoring script for Torch Serve was relatively simple, essentially loading the model and doing a forward pass. TorchServe's behavior was predictable, and debugging the PyTorch model in the Azure environment was easier in some respects (since we could interact with the model directly in Python and get immediate error messages). The custom environment did require a bit more work

(managing the correct PyTorch version and dependencies), but once set up, TorchServe handled requests without issue. In summary, TorchServe proved to be a solid choice for serving the image model, achieving slightly better latency and equal throughput compared to TensorFlow in our tests.

B. TensorFlow Serving

Performance on RNN: TensorFlow Serving excelled for the RNN text model, particularly as concurrency increased. It maintained low latency where TorchServe’s latency grew, indicating better internal optimization for this kind of workload. One contributing factor could be that TensorFlow’s runtime can parallelize operations or handle vectorized batch computations more effectively for the RNN (even though we sent single requests, the internals might utilize threads or SIMD for embedding lookups, etc.).

Deployment Considerations: On the flip side, setting up TensorFlow Serving in Azure was aided by the curated environment (no need to build from scratch), but we encountered several compatibility issues – for example, ensuring the Saved Model had the right signature and data types for CPU, and adjusting the input processing in the script. The error messages (such as the `_UserObject...predict` and dtype mismatches) required careful fixes. Once resolved, the TensorFlow endpoint was robust and did not require further tweaking, and it benefited from the highly optimized TensorFlow engine. Integration with Azure is also a plus for TensorFlow: using a curated environment meant less custom setup and the Azure ML system handled scaling and containerization of the TF serving components.

C. Comparison and Other Observations

In terms of latency consistency, after initial adjustments, both frameworks showed stable latency with very small variance. We did not observe significant latency spikes even at heavy load, suggesting that both the TensorFlow runtime and PyTorch runtime effectively managed the queue of requests. The slight latency advantage of one framework in one scenario did not come with volatility – e.g., TorchServe’s latencies for the CNN were consistently around 30-40ms without outliers, and TensorFlow’s latencies for the RNN stayed very consistently low. Another observation is that TorchServe was easier to work with for quick iteration – since it’s just PyTorch under the hood in our setup, we could easily modify the Python code, whereas TensorFlow’s model loading felt more like a black-box that required the correct export. However, Azure’s support for TensorFlow (through curated environments and examples) is more mature, meaning a lot of configurations (like GPU drivers, etc.) were handled for us when using TF. We also note that both frameworks in this test were CPU-bound; with GPU deployment, the performance differences might shift, and additional configuration (like ensuring TorchServe uses CUDA, or TensorFlow Serving uses a GPU device) would be needed.

In summary, TorchServe vs TensorFlow Serving is not a one-size-fits-all decision. The choice can depend on the type of model and the deployment context:

- For computer vision models (CNNs), TorchServe (using PyTorch) showed a slight edge in our latency tests. If a team is already using PyTorch for development, TorchServe allows seamless deployment with minimal performance penalty and even a minor gain in response time for CPU inference.
- For NLP or sequential models (RNNs), TensorFlow Serving demonstrated superior performance on CPU, which might make it a better choice especially if the models are originally developed in TensorFlow or if maximum throughput on limited hardware is critical. The optimized TensorFlow runtime can be advantageous for such tasks.

IX. CONCLUSION

This project conducted a thorough head-to-head comparison of TensorFlow Serving and TorchServe on Azure for two representative models. The results show that each framework has its strengths. For image classification tasks (like our CNN on Fashion MNIST), TorchServe slightly outperformed TensorFlow Serving in terms of latency, while matching it in throughput. This suggests PyTorch’s serving can be highly efficient for vision workloads. In contrast, for text analysis tasks (like our RNN sentiment model), TensorFlow Serving provided faster inference and handled heavy loads more robustly on CPU. The TensorFlow-based deployment needed more careful preparation (ensuring model compatibility), but rewarded us with excellent performance.

When deciding between the two frameworks, one should consider the nature of the model and the ecosystem: If the model is developed in PyTorch and especially if it will be deployed for image or other vision tasks, TorchServe is an excellent choice, offering straightforward model export and competitive performance. If the model is in TensorFlow or the use case involves sequence processing and you require maximal efficiency on CPU, TensorFlow Serving might be preferable. It is also worth noting that both frameworks achieved the same level of throughput at saturation in our tests – implying that either can fully utilize available hardware up to its limits.

From an operational standpoint, using Google Colab for training and Azure for deployment (our project workflow) proved to be cost-effective and convenient. We trained models in a free environment and then seamlessly deployed to Azure’s managed service for serving. This separation of training and serving environments is a practical approach for many real-world projects.

In conclusion, both TensorFlow Serving and TorchServe are capable MLOps solutions, and the optimal choice depends on the specific context. Organizations with existing PyTorch models and a need for quick deployment might lean towards TorchServe, while those requiring the maturity and optimization of TensorFlow (especially for certain model types) might choose TensorFlow Serving. Importantly, the engineering effort to get models running was manageable for both, and Azure ML’s platform provided a neutral ground to compare them fairly. We recommend TorchServe for slightly better latency in vision inference and TensorFlow Serving for its strength in handling text models and integration

convenience. Each can be used to build scalable, robust inference services when configured properly.

X. REFERENCE

- [1] TensorFlow Serving Official Documentation. [Online]. Available: <https://www.tensorflow.org/tfx/guide/serving>
- [2] TorchServe Official Documentation. [Online]. Available: <https://pytorch.org/serve/>
- [3] Azure ML Online Endpoints. Microsoft Azure Documentation. [Online]. Available: <https://learn.microsoft.com/azure/machine-learning/how-to-deploy-online-endpoints>
- [4] Fashion MNIST Dataset. (2017). [Online]. Available: <https://github.com/zalandoresearch/fashion-mnist>
- [5] T. Hafeez, A. Anwar, and A. R. Butt, "Performance Comparison of TensorFlow Serving and TorchServe for Deep Learning Models," arXiv preprint arXiv:2104.07839, 2021.
- [6] Y. Zhang, L. Liu, and D. Gmach, "A Machine Learning-Based Resource Optimization Framework for Cloud Machine Learning Services," in Proceedings of the IEEE International Conference on Services Computing (SCC), San Francisco, CA, USA, 2016, pp. 292–299.
- [7] N. Kumar, S. P. Singh, and P. K. Gupta, "Performance Evaluation of Cloud-Based Machine Learning Deployment Architectures," in Proceedings of the IEEE International Conference on Advances in Computing, Communications and Informatics (ICACCI), Bangalore, India, 2018, pp. 2176–2181.
- [8] H. Zhang et al., "Evaluating Inference Serving Systems for Machine Learning Models in Production," IEEE Transactions on Services Computing, vol. 15, no. 2, pp. 651–664, 2022.